

AI Course

Capstone Project

Final Report

For students (instructor review required)

Bayan: An On-Device Arabic Reading Assistant for Readers with Dyslexia

Date	22 September 2026
Team Name	Cogni
Members	Marwan Elamami (Team Leader) Abdulrahman Khengari Ahmed Alaeb Sanad Ali Mohammed Thabet Abdul Majid

Abstract

Arabic places barriers in front of readers with dyslexia that tools built for Latin scripts do not address: short vowels are omitted, clitics attach directly to the stem, and clauses chain without sentence-final punctuation. We present Bayan, an Android plugin that simplifies Arabic text in place. Selecting text in any application offers تبسيط (*Simplify*) in the system text-selection menu; simplification, diacritization and read-aloud all run on the device, so the text never leaves the phone. This report covers the work completed at the two-thirds point of a three-week capstone. Our central methodological finding is that Arabic simplification corpora contain so many sentences needing no change that a model doing nothing scores well: on the SAMER test split, copying the input yields SARI 77.5 overall but 56.1 on the rows a human actually simplified. Reported against that baseline, our first fine-tuned model gains +2.8 SARI overall and +7.5 where the work is — and *loses* to the baseline under the greedy decoding we ship on-device. We also detect and remove 9.3% test-set contamination in a published Arabic simplification corpus before training on it. Every claim below is labelled [measured], [decided] or [planned].

1. Introduction

1.1. Background Information

Dyslexia impairs reading through phonological processing. Modern Standard Arabic compounds this in three specific ways.

Omitted short vowels. Written Arabic normally leaves short vowels unmarked, so one orthographic form carries several readings: كتب is *kataba* (he wrote), *kutiba* (it was written) or *kutub* (books). The reader resolves this from context — precisely the process dyslexia impairs.

Agglutination. Arabic is templatic and agglutinative. Conjunctions, articles and object pronouns attach directly to the stem, producing long unbroken words that crowd the line and resist chunking.

Clause chaining. Prose chains clauses with و and ف across long stretches without sentence-final punctuation, so a single sentence can exceed working memory.

Full diacritization is not a complete answer: marking every character adds visual clutter of its own. The useful intervention is to simplify first, then vocalise. We found no deployed Arabic dyslexia-support software and no large open Arabic simplification corpus, which shapes both the product and the data work below.

1.2. Motivation and Objective

Objective. Let an Arabic reader with dyslexia read difficult text independently, wherever they meet it.

[decided] The product is an Android plugin registered on the system PROCESS_TEXT intent, so تبسيط (*Simplify*) appears in the text-selection menu of every application. The reader selects text in a message, a news page or a PDF and is helped in place. A standalone application was rejected for the opposite reason: it reintroduces the copy-and-paste friction the project exists to remove.

[decided] Every model runs on the device. The reader's text never leaves the phone, the tool works offline, and no hosted service sits on the critical path of the final demonstration. Figure 1 shows the arrangement.

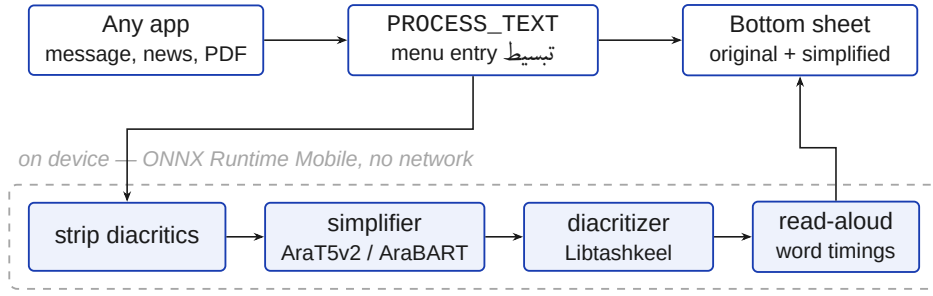


Figure 1: System architecture. The plugin is entered from the host application's text-selection menu and returns a bottom sheet over it. All three models run locally through ONNX Runtime Mobile. Diacritics are stripped *before* the simplifier and restored after, for the reason measured in subsection 3.3.

1.3. Members and Role Assignments

Table 1: Role assignments.

Member	Responsibility
Marwan Elamami	Team lead; architecture; compression and quantization; integration; human evaluation
Abdulrahman Khengari	Data lead; simplification corpus; readability classifier; annotation guidelines and agreement
Ahmed Alaeb	Model training lead; corpus conversion; training infrastructure
Sanad Ali	Diacritization selection and benchmarking; co-training the simplification models
Mohammed Thabet	Android plugin; on-device inference integration; read-aloud with word highlighting
Abdul Majid	Evaluation lead; test-set design and locking; leakage detection; scoring pipeline

1.4. Schedule and Milestones

Week 1 (16–20 Sep), complete: data pipeline and validators; readability classifier; test-set design; product-form decision; application skeleton. Week 2 (21–27 Sep), in progress: corpus generation and review; first and second training runs; diacritization benchmark; locked test sets and scoring pipeline; Android plugin with on-device inference. Week 3 (28 Sep–4 Oct): architecture frozen; int8 quantization with size and latency measured on a real device; annotator agreement; full benchmark including human error annotation; human evaluation with dyslexic readers. Week 4 (5–6 Oct): submission and pitch.

Work is tracked package by package in the Work Breakdown Structure: 8 phases, 38 work packages, one named owner each.

2. Project Execution

2.1. Data Acquisition

Table 2 lists the five sources, all under licences permitting our use. SAMER is used under written approval from CAMEL Lab to fine-tune and to publish the resulting weights non-commercially; the corpus itself is never redistributed.

2.2. Training Methodology

[measured] Model 1. AraT5v2-base-1024 [6], 368M parameters, fine-tuned on SAMER L5→L3. Learning rate 10^{-4} , AdamW with weight decay 0.01, 100 warmup steps, effective batch 16, fp16,

Table 2: Corpora. Counts are usable pairs after filtering.

Source	Size	Licence	Role
BAREC [1]	69,441 sent.	CC BY-SA 4.0	source sentences; readability labels
SAMER [2]	14,343 pairs	research, non-redistributable	literary simplification pairs
Baseet [3]	31,104 pairs	unlicensed (permission sought)	general domain, three levels
DAASI [4]	1,509 + 350	CC0	human-written; out-of-domain test
Bayan corpus (ours)	in progress	ours	LLM-generated behind a human gate
Tashkeela [5]	–	open	diacritization validation

gradient checkpointing, 10 epochs over 3,250 steps, seed 42. Identity pairs capped at 20% in training and left untouched in dev and test.

Three corrections follow from its measured behaviour and apply to every later run.

1. **Select on the changed-reference subset, decoded greedily.** Selecting on full-dev SARI pays a model for inaction, because roughly half the dev set needs no change; selecting under beam search measures a decoder we do not ship.
2. **Stop on validation loss, not SARI.** Validation loss bottoms near step 2,200 while SARI wanders inside noise for a further 1,400 steps (Figure 4) and picked its checkpoint on a 0.18 difference.
3. **Train on plain and diacritized copies of every pair.** SAMER carries no diacritics at all, and the resulting model degrades badly on diacritized input (Figure 5).

[planned] Model 2. The same recipe over a mixed corpus of $\approx 107,000$ pairs across three domains, with level tokens so one model offers three simplification strengths plus a leave-unchanged mode. AraBART [7] (139M) is trained alongside AraT5v2 on identical data; the choice between them will be made on measured results, with the smaller model preferred at equal quality because it makes on-device decoding affordable.

2.3. Workflow

Figure 2 shows the path from corpus to score. Two gates decide what may be trained on: a human review of every LLM-generated pair, and a leakage check against the locked test sets. Only the evaluation lead runs the locked sets, and only on final models; everyone else tunes on dev.

2.4. System Design

One Android application, two activities, three models. `ProcessTextActivity` registers `android.intent.action.TEXT` with mime type `text/plain`. A translucent theme lets the result float over the host application, which avoids requesting the `SYSTEM_ALERT_WINDOW` overlay permission entirely — a meaningful reduction in what the user must grant. Inference runs behind a `TextSimplifier` interface so the engine can be replaced without touching the interface; decoding is greedy and the tokenizer is exported inside the ONNX graph, so no Python runtime is needed on the phone.

[decided] Size budget: 250 MB for the simplifier plus ≈ 45 MB for diacritization and read-aloud. Vocabulary-pruned int8 is ≈ 223 MB and fits; int4 (≈ 112 MB) is the fallback if load time or memory on a mid-range phone proves binding. Models download on first run rather than shipping inside the APK.

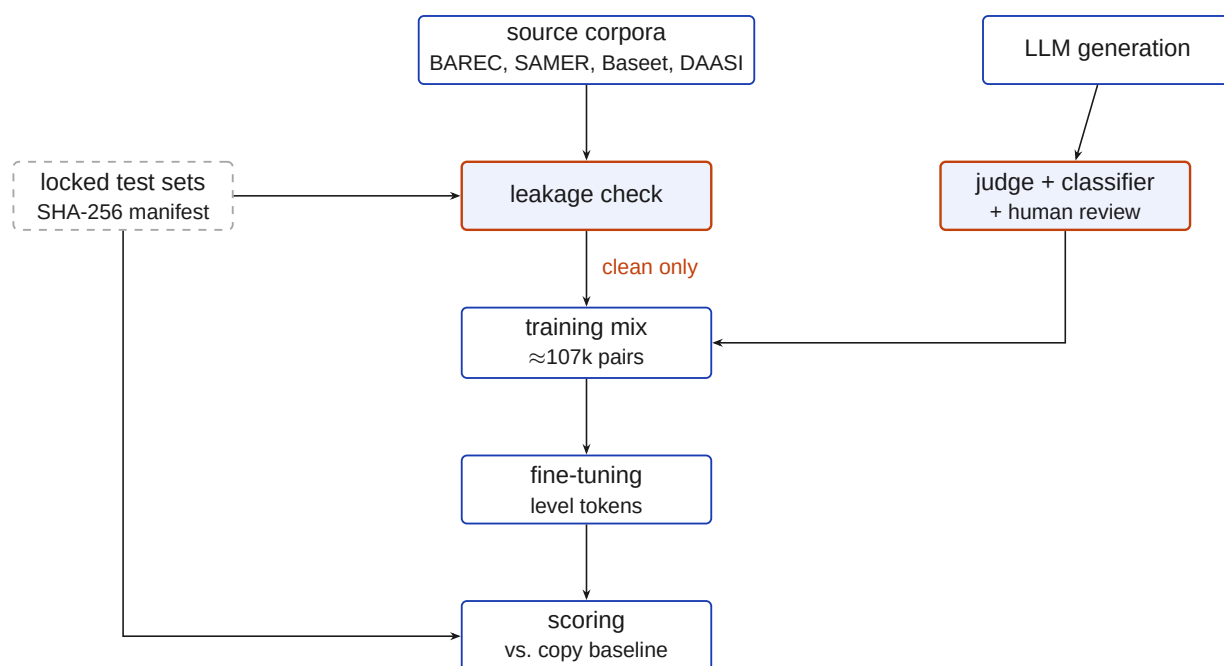


Figure 2: Data and evaluation workflow. The two orange gates are the ones that can reject data: human review of generated pairs, and the leakage check that removed 9.3% of one published corpus.

3. Results

3.1. Data Preprocessing

Normalisation for comparison only. Diacritics and punctuation are stripped and alef variants (آ إ أ), ؤ/و and اى/ي unified. This is used for leakage detection and scoring alignment, never on training text, where the original orthography is preserved.

Test sets locked first. Two splits were locked before any training and are never trained on: BAREC test (7,286 rows) and SAMER test (3,277 rows), each recorded by row count and SHA-256 in a committed manifest. Both are the official upstream splits, unmodified.

[measured] Leakage filtering. Every training file is checked against both locked sets for exact matches and near-duplicates, using character 5-gram Jaccard overlap at a 0.8 threshold. Matches shorter than five words are reported but not counted, because short formulaic fragments recur in any corpus.

This check found that **3,190 of Baseet’s 34,294 rows (9.3%)** overlapped our locked test sets — 1,829 touching SAMER test and 1,361 touching BAREC test. Put the other way, **1,835 of SAMER’s 3,933 substantive test sentences (47%) appear somewhere in that corpus.** Training on it unfiltered would have produced inflated scores with no error and nothing to notice. The rows were removed and the remainder re-verified clean.

3.2. Exploratory Data Analysis

The corpora differ in ways that turned out to drive model behaviour, so we report them rather than assume them.

SAMER is literary and short. Drawn from early twentieth-century novels; the test split has median sentence length 6 words, and 17.8% of rows are three words or fewer — chapter headings and dialogue fragments rather than sentences. 48.8% of test rows are identity pairs. It splits by

chapter, not by novel, so all fifteen novels appear in all three splits: its test set is fully in-domain and SARI on it reads optimistically. It contains **no diacritics at all**.

BAREC duplicates itself. 311 sentences appear in both its train and test splits, and 797 of 7,286 test rows are two words or fewer. Both cause false alarms in leakage checking and motivate the minimum-length floor. 56% of its sentences carry diacritics.

Baseet is closer to the product. Median source length 14 words, 35.4% diacritized, no identity pairs at any level; its strongest level shortens a 14-word median source to 9 words. **DAASI** is government and insurance text, median 12 words, 32% diacritized, no identity pairs.

3.3. Modeling

[measured] The copy baseline comes first. On a corpus where half the sentences need no change, a model that does nothing scores well, so every result is reported beside copying the input. On SAMER test the baseline is **SARI 77.5 overall** but **56.1** on the 1,678 rows a human actually simplified. The second number is the bar.

[measured] Model 1. On SAMER dev: SARI 77.06 overall and 63.66 on changed references, against a baseline of 74.25 and 56.21 — +2.81 and +7.46. Under greedy decoding, which is what the on-device build uses, the same model scores *below* the baseline (Figure 3). Beam search accounts for the difference; whether to pay its cost on the device is pending a latency measurement.

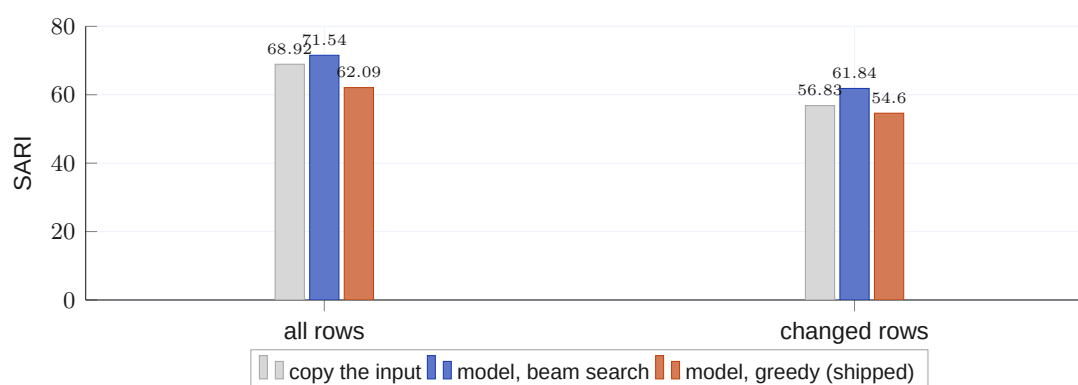


Figure 3: SARI on 300 matched SAMER dev rows, same model, same rows. Reported against a copy-the-input baseline, the gain is modest; under the greedy decoder we ship, it is negative. Scored with the evaluate SARI implementation [8], reproduced independently.

[measured] Behavioural findings. The model copies its input 66.5% of the time against a human rate of 41.2%, and returns unchanged text on 46.8% of the rows a human simplified. Where it edits, 75.3% of edits change a single word (humans: 53.2%), and the median prediction-to-source length ratio is **1.000** — it substitutes vocabulary but never restructures a sentence. Figure 4 shows why: the selection metric flattened while the behaviour we care about was still improving.

[measured] Out-of-domain collapse. On diacritized BAREC input, which the training data never contained, only 60% of source words survive into the output and 38% of sentences lose more than half their content to character-level corruption; on undiacritized input the figures are 90% and 3% (Figure 5). This is why diacritization must run *after* simplification in Figure 1, never before.

[measured] Meaning errors are not rare. 832 of the model's substitutions disagree with the human reference, 347 of them on words the human deliberately left alone. Observed examples include عزلة (isolation) → استعباد (enslavement) and ملجم (bridled) → مسلح (armed). Neither SARI nor BLEU detects this class of error, which is why BERTScore [9] and human error annotation are part of the evaluation.

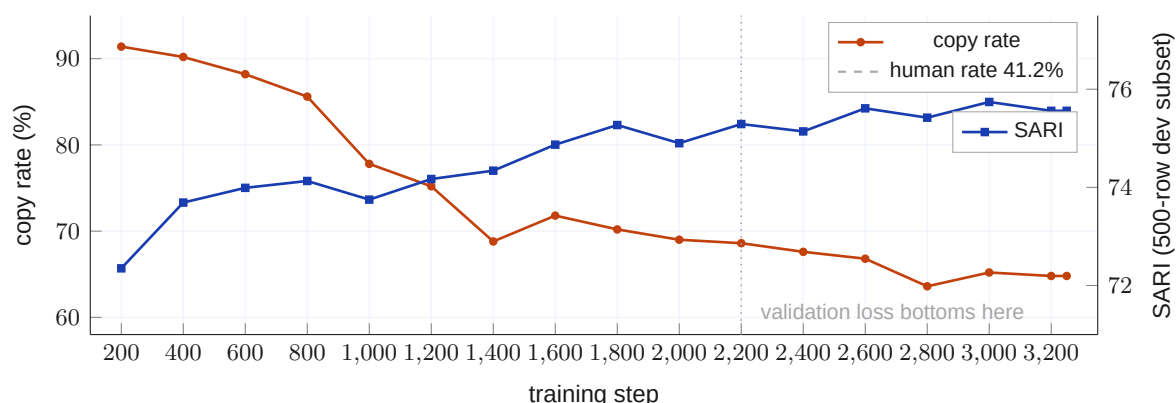


Figure 4: Training dynamics for model 1. Copy rate falls monotonically from 91.4% and has not converged at the final step, while SARI — the metric that selected the checkpoint — is flat inside noise after step 1,800. Validation loss bottoms near step 2,200 and then rises.

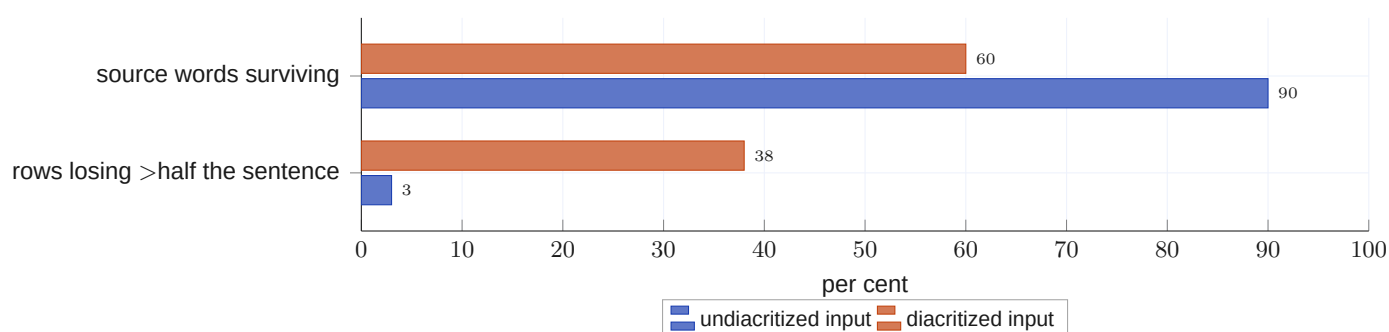


Figure 5: Out-of-domain behaviour on 300 BAREC test sentences, greedy decoding. The model was trained on SAMER, which contains no diacritics; 56% of BAREC does.

[measured] Diacritization. Four models were benchmarked on the pinned Tashkeela test split (Table 3). Libtashkeel was selected. CATT is statistically tied on accuracy — the 0.01 point difference is about six character positions across 67,576 — but sixteen times larger and six times slower. CAMEL Tools was rejected on a stronger ground than accuracy: it altered base letters in 32% of sentences, corrupting text before the simplifier sees it.

Table 3: Diacritization benchmark, 743 sentences from the pinned Tashkeela/CATT test fixture. DER excludes case endings in the second column.

Model	DER	DER ^{—case}	WER	Base-letter	Size	Licence
CAMEL Tools MLE [10]	16.57%	13.97%	56.73%	32.0%	129 MB	MIT/GPL
CATT [11]	8.44%	6.92%	29.40%	0.0%	77.9 MB	CC BY-NC
Mishkal [12]	12.74%	10.12%	39.33%	0.0%	4.3 MB	GPLv3
Libtashkeel [13]	8.43%	6.91%	29.57%	0.0%	4.8 MB	MIT

3.4. User Interface

Selecting Arabic text in any application shows تبسيط in the system menu. Choosing it opens a modal bottom sheet over the host application with the original text in a muted card and the simplified text below, plus controls for read-aloud, copy and settings.

Typography is set for dyslexic reading rather than density: line height 1.6, additional line spacing 10 sp, letter spacing 0.04, and right-to-left text left *unjustified*, because justification in Arabic stretches words and opens uneven rivers of space. Selection behaviour was verified across Chrome, WhatsApp, Google Keep, Gmail and PDF readers.

3.5. Testing and Improvements

41 unit and Robolectric tests cover intent extraction, empty-input handling, view binding and the typography attributes. The leakage checker is tested against a deliberately planted duplicate and must exit non-zero. Running the scoring script end to end on real data — rather than only on fixtures — exposed a bug that placed every BAREC row in the unchanged-reference bucket instead of a separate no-reference bucket, since BAREC has no gold simplifications at all.

Improvements already made as a result of measurement: the copy baseline is reported beside every score; checkpoint selection moved to the changed-reference subset under greedy decoding; diacritics are stripped before the simplifier; and 9.3% of a published corpus was removed before it could contaminate our results.

4. Projected Impact

4.1. Accomplishments and Benefits

[measured] To date: a copy-the-input baseline on two locked test sets; a first model trained and scored against it with its failure modes characterised; a four-model diacritization benchmark; and 9.3% test-set contamination detected and removed from a published corpus before training.

Built: a working Android plugin shell on the `PROCESS_TEXT` intent; a locked and hashed test-set manifest; a reusable leakage checker; and a scoring pipeline that reports a copy baseline beside every metric.

Benefits. Reading independence at the moment and place the difficulty occurs, rather than in a separate application. Support for students, teachers and schools, and for readers with low literacy and learners of Arabic. Everything runs on the device, so the tool works offline and the reader's text stays private.

For the field: an Arabic simplification corpus, an evaluation benchmark with human-written references, and a leakage-detection tool, none of which existed in openly reusable form.

4.2. Future Improvements

[planned] Before submission: complete the mixed-corpus runs and choose between AraT5v2 and AraBART on measured results; quantize the winner and measure size and latency on a real phone; run the human evaluation with Arabic readers who have dyslexia; report annotator agreement.

Known limitations. The current model substitutes vocabulary but does not restructure sentences, and clause splitting is arguably the larger part of the problem for dyslexic readers. Automatic metrics do not detect meaning-altering substitutions, so human annotation remains necessary. On-device latency is not yet measured, so the choice between greedy and beam decoding is open.

Beyond the capstone. Fine-tune the diacritizer on simplified output rather than using it unchanged; add read-aloud with word-level timing; and extend the corpus beyond the three domains currently covered.

5. Team Member Review and Comment

Each member writes their own review before submission.

Name	Review and comment
Marwan Elamami	
Abdulrahman Khengari	
Ahmed Alaeb	
Sanad Ali	
Mohammed Thabet	
Abdul Majid	

5. References

- [1] CAMEL Lab. *BAREC: Balanced Arabic Readability Evaluation Corpus*, 2025. 69,441 sentences with 19-level readability labels, CC BY-SA 4.0.
- [2] B. Alhafni, M. Al Khalil and N. Habash. The SAMER Arabic Text Simplification Corpus. *LREC-COLING*, 2024.
- [3] Baseet: A Multi-Level Dataset and Hybrid Framework for Arabic Text Simplification. University of Jeddah graduation project. <https://github.com/raya-y/Baseet>
- [4] DAASI: Arabic simplification pairs for government and insurance text. CC0; 1,509 pairs and a 350-pair held-out split.
- [5] T. Zerrouki and A. Balla. Tashkeela: Novel corpus of Arabic vocalized texts. *Data in Brief*, 2017.
- [6] E. M. B. Nagoudi, A. Elmadany and M. Abdul-Mageed. AraT5: Text-to-Text Transformers for Arabic Language Generation. *ACL*, 2022.
- [7] M. K. Eddine, N. Tomeh, N. Habash, J. Le Roux and M. Vazirgiannis. AraBART: a Pretrained Arabic Sequence-to-Sequence Model for Abstractive Summarization. *WANLP*, 2022.
- [8] W. Xu, C. Napoles, E. Pavlick, Q. Chen and C. Callison-Burch. Optimizing Statistical Machine Translation for Text Simplification. *TACL*, 4:401–415, 2016.
- [9] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger and Y. Artzi. BERTScore: Evaluating Text Generation with BERT. *ICLR*, 2020.
- [10] O. Obeid et al. CAMEL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing. *LREC*, 2020.
- [11] abjadai. CATT: Character-based Arabic Tashkeel Transformer, 2024. <https://github.com/abjadai/catt> (fixture pinned at commit 8d5330499feb).
- [12] T. Zerrouki. Mishkal: Arabic text vocalization software. <https://github.com/linuxscout/mishkal>
- [13] Libtashkeel: an ONNX Arabic diacritization model. MIT licence, 4.79 MB; evaluated through the `text2tashkeel` wrapper.